

10. offline フルレース simulation 本体

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	scripts/solar_sim.py
種別	Python
区分	offline core
役割	profile、forecast、route、maps を使って全レースを逐次再生し、upper/lower 相当の実行を CSV と HTML に落とす。
起動文脈	simulate や historical-simulate で直接呼ばれる同期版の基準実装。

このファイルを読む理由

profile、forecast、route、maps を使って全レースを逐次再生し、upper/lower 相当の実行を CSV と HTML に落とす。

- SolarCarModel を直に持つ同期版なので、数理解の最短入口。
- full summary JSON、detail CSV、upper plan CSV、HTML report を生成する。
- live の mpc_node とかなり同型の距離上位計画ロジックを持つ。

呼び出し関係

どこから呼ばれるか

- scripts/solar_control.sh

次に読むと理解が進むファイル

- mpc_solarcar/model.py
- mpc_solarcar/upper_horizon.py
- mpc_solarcar/upper_solver.py

同一リポジトリ内の主要依存

- mpc_solarcar/model.py
- mpc_solarcar/route_utils.py

- `mpc_solarcar/schedule_utils.py`
- `mpc_solarcar/signal_utils.py`
- `mpc_solarcar/solar_profile.py`
- `mpc_solarcar/upper_cost.py`
- `mpc_solarcar/upper_horizon.py`
- `mpc_solarcar/upper_policy.py`
- `mpc_solarcar/upper_solver.py`

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
os.makedirs(parent, exist_ok=True)

class DetailCsvStream:
    """Write the 1 Hz execution trace without retaining the full race in RAM."""

    def __init__(self, path):
        self.path = str(path)
        self._file = None
        self._writer = None
        self.fieldnames = []
        self.row_count = 0

    def write(self, row):
        if self._writer is None:
            ensure_parent_dir(self.path)
            self.fieldnames = list(row.keys())
            opener = gzip.open if self.path.lower().endswith('.gz') else open
            self._file = opener(self.path, 'wt' if opener is gzip.open else 'w', encoding='utf-8', newline='')
            self._writer = csv.DictWriter(self._file, fieldnames=self.fieldnames, extrasaction='ignore')
            self._writer.writeheader()
            self._writer.writerow(row)
```

主要構造の読み取り

主要クラスは `DetailCsvStream`。主要関数は `load_yaml`, `sim_log`, `select_optimized_vector`, `limit_step_duration_to_distance`, `terminal_soc_predictions`, `advance_rate_limiter_to_distance_boundary`, `snap_execution_stop_speed_kmh`, `choose_integration`。CLI 引数宣言は 96 件。

主要クラス・関数

行	種別	名前	補足
549	class	DetailCsvStream	
63	function	load_yaml	args: path
73	function	sim_log	args: message
77	function	select_optimized_ vector	args: res, x0
115	function	limit_step_duration_ to_distance	args: step_sec, speed_kmh, s0_km, race_km
129	function	terminal_soc_ predictions	args: upper_solve_log
145	function	advance_rate_limiter_ to_distance_boundary	args: limiter, target_kmh
195	function	snap_execution_stop_ speed_kmh	args: speed_kmh, target_kmh
212	function	choose_integration_ step_seconds	args: simulation_step_sec, weather_step_sec
218	function	choose_stationary_ integration_step_ seconds	args: simulation_step_sec, weather_step_sec, requested_step_sec
230	function	write_model_snapshot	args: detail_csv, parameters, map_paths
278	function	get_workspace_ revision	args: root_dir
331	function	timestamp_ns	args: value
335	function	load_stops	args: path
365	function	load_profile	args: path
373	function	forecast_distance_ column	args: df
383	function	load_forecast_ dataframe	args: path, tzname
410	function	merge_forecast_ dataframes	args: primary, fallback
423	function	build_forecast_grid_ payload	args: df
476	function	interp_forecast_grid	args: payload, col, t_utc, s_km, default
523	function	load_progress_ reference_dataframe	args: path
543	function	ensure_parent_dir	args: path
552	function	__init__	args: self, path
559	function	write	args: self, row
570	function	close	args: self
576	function	__enter__	args: self
579	function	__exit__	args: self, exc_type, exc, tb
583	function	_deep_copy_cfg	args: cfg
587	function	_set_nested	args: cfg, dotted_key, value
601	function	apply_overrides	args: cfg, overrides
618	function	build_config_tag	args: profile_path, profile_cfg, overrides
633	function	tag_output_path	args: path_value, tag, default_ext
645	function	apply_profile_cfg_to_ args	args: profile_path, profile_cfg, args

798	function	apply_profile_defaults	args: args
806	function	resolve_config_path	args: profile_path, value
820	function	evaluate_model_validation_gate	args: cfg, profile_path
1032	function	get_profile_val	args: df, s_km, field, default
1040	function	parse_utc_arg	args: raw_value
1052	function	resolve_forecast_mode	args: mode, df
1064	function	forecast_row_index	args: df, sim_t
1082	function	format_metric	args: value, digits, default
1094	function	decimate_xy	args: xs, ys, max_points
1105	function	build_svg_chart	args: xs, ys
1146	function	flatten_params_for_report	args: cfg
1149	function	visit	args: prefix, value
1166	function	write_simulation_report	args: path, summary, detail_df, params_rows
1288	function	mpc_solve	args: model, data, z0, Tb0, s0_km, v0_kmh, v_max_kmh, term_soc_min, w_dv, w_dv_limit, dv_max_kmhps, w_T, w_speed_limit, w_current, speed_profile, soc_target, soc_band, w_soc_target, w_soc_band, schedule, soc_day_end_max, w_soc_day_max, soc_finish_target, soc_finish_tol, w_soc_progress, w_soc_terminal, race_km, soc_day_end_target, soc_day_end_tol, w_soc_day_track
1305	function	quad_penalty	args: x, cap
1312	function	cost	args: v
1402	function	soc_guard_speed	args: model, v_kmh, z, Tb, s_km, d0, route_profile, mode, soc_guard
1409	function	z_next_for	args: v_kmh_local
1451	function	soc_day_guard_speed	args: model, v_kmh, z, Tb, s_km, d0, route_profile, target_soc, tol
1458	function	z_next_for	args: v_kmh_local
1485	function	main	
1852	function	integration_step_seconds	
1859	function	forecast_has_coverage	args: t_utc
1868	function	integrate_drive_time_between	args: t_start_utc, t_end_utc
1885	function	reference_value_at_time	args: t_utc, field
1907	function	reference_distance_at_time	args: t_utc
1958	function	time_to_index	args: dt_utc
2095	function	remaining_day_budget	args: sim_t_local, _k_idx

ファイルを上から読んだときの定義順

行	内容
23	<code>_numeric_threads</code> に <code>str(os.environ.get('SOLAR_NUMERIC_THREADS', '1'))</code> の結果を代入する。
24	<code>('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS', 'NUMEXPR_NUM_THREADS')</code> を順に走査し、各要素を <code>_thread_variable</code> に入れて処理する。
36	<code>ROOT</code> に <code>Path(__file__).resolve().parents[1]</code> の結果を代入する。
37	条件 <code>str(ROOT) not in sys.path</code> を判定し、真なら内部処理を行う。
60	<code>DISTANCE_EPS_KM</code> に <code>1e-06</code> の結果を代入する。
63	関数 <code>load_yaml</code> を定義する。
73	関数 <code>sim_log</code> を定義する。
77	関数 <code>select_optimized_vector</code> を定義する。
115	関数 <code>limit_step_duration_to_distance</code> を定義する。
129	関数 <code>terminal_soc_predictions</code> を定義する。
145	関数 <code>advance_rate_limiter_to_distance_boundary</code> を定義する。
195	関数 <code>snap_execution_stop_speed_kmh</code> を定義する。
212	関数 <code>choose_integration_step_seconds</code> を定義する。
218	関数 <code>choose_stationary_integration_step_seconds</code> を定義する。
230	関数 <code>write_model_snapshot</code> を定義する。
278	関数 <code>get_workspace_revision</code> を定義する。
331	関数 <code>timestamp_ns</code> を定義する。
335	関数 <code>load_stops</code> を定義する。
365	関数 <code>load_profile</code> を定義する。
373	関数 <code>forecast_distance_column</code> を定義する。
383	関数 <code>load_forecast_dataframe</code> を定義する。
410	関数 <code>merge_forecast_dataframes</code> を定義する。
423	関数 <code>build_forecast_grid_payload</code> を定義する。
476	関数 <code>interp_forecast_grid</code> を定義する。
523	関数 <code>load_progress_reference_dataframe</code> を定義する。
543	関数 <code>ensure_parent_dir</code> を定義する。
549	クラス <code>DetailCsvStream</code> を定義する。
583	関数 <code>_deep_copy_cfg</code> を定義する。
587	関数 <code>_set_nested</code> を定義する。
601	関数 <code>apply_overrides</code> を定義する。
618	関数 <code>build_config_tag</code> を定義する。
633	関数 <code>tag_output_path</code> を定義する。
645	関数 <code>apply_profile_cfg_to_args</code> を定義する。
798	関数 <code>apply_profile_defaults</code> を定義する。
806	関数 <code>resolve_config_path</code> を定義する。
820	関数 <code>evaluate_model_validation_gate</code> を定義する。
1032	関数 <code>get_profile_val</code> を定義する。
1040	関数 <code>parse_utc_arg</code> を定義する。
1052	関数 <code>resolve_forecast_mode</code> を定義する。
1064	関数 <code>forecast_row_index</code> を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
2	<code>importargparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L1486, L1534。
3	<code>importcsv</code>	CSV の逐次読込・逐次書込を行うため。このファイル内での主な使用位置は L565。
4	<code>importcopy</code>	設定辞書や <code>payload</code> を安全に複製するため。このファイル内での主な使用位置は L584。
5	<code>importgzip</code>	detail CSV などの圧縮出力を行うため。このファイル内での主な使用位置は L563, L564。
6	<code>importhashlib</code>	snapshot ID や入力資産の <code>digest</code> を作るため。このファイル内での主な使用位置は L238, L252, L624。
7	<code>importhtml</code>	HTML report の文字列を安全に埋め込むため。このファイル内での主な使用位置は L1131, L1136, L1206, L1210, L1214, L1222, L1253, L1254。
8	<code>importjson</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L253, L1206, L2830, L2950, L4001, L4005。
9	<code>importmath</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L134, L138, L885, L886, L887, L888, L889, L890, ...。
10	<code>importtime</code>	wall/monotonic time に基づく周期制御や <code>freshness</code> 判定を行うため。このファイル内での主な使用位置は L2055, L3713。
11	<code>importos</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L23, L30, L544, L546, L626, L639, L739, L742, ...。
12	<code>importsubprocess</code>	git 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L286, L299, L308, L317。
13	<code>importsys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L37, L38, L4028。
14	<code>importtraceback</code>	例外時に <code>crash log</code> を残すため。このファイル内での主な使用位置は L4025。
15	<code>fromcollectionsimportdeque</code>	固定長の時系列や遅延キューを効率よく保持するため。このファイル内での主な使用位置は L2075。
16	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L36, L235, L270, L278, L810, L813, L835, L901, ...。
17	<code>fromzoneinfoimportZoneInfo</code>	zoneinfo から <code>ZoneInfo</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L393, L2030, L2032, L3736, L3756。
18	<code>fromdatetimeimportdatetime, timedelta, timezone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L476, L626, L628, L1046, L1048, L1049, L1064, L1364, ...。
32	<code>importnumpyasnp</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L78, L93, L100, L135, L433, L470, L484, L486, ...。

33	<code>import pandas as pd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L332, L369, L373, L374, L383, L384, L387, L410, ...。
34	<code>import yaml</code>	<code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L68, L271, L612, L623, L3785。
40	<code>from mpc_solarcar. model import SolarCarModel, Params</code>	車体物理・電気モデル本体から <code>SolarCarModel</code> 、 <code>Params</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/model.py</code> 。このファイル内での主な使用位置は L1689, L1703。
41	<code>from mpc_solarcar.route_ utils import average_profile, interpolate_profile</code>	<code>route_utils.py</code> から <code>average_profile</code> 、 <code>interpolate_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/route_utils.py</code> 。このファイル内での主な使用位置は L1035, L2460, L2656。
42	<code>from mpc_solarcar.schedule_ utils import DriveSchedule</code>	<code>schedule_utils.py</code> から <code>DriveSchedule</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/schedule_utils.py</code> 。このファイル内での主な使用位置は L1664。
43	<code>from mpc_solarcar.signal_ utils import SmoothRateLimiter</code>	<code>signal_utils.py</code> から <code>SmoothRateLimiter</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/signal_utils.py</code> 。このファイル内での主な使用位置は L2078。
44	<code>from mpc_solarcar.solar_ profile import get_path, get_ section, load_profile as load_ workflow_profile</code>	<code>profile</code> YAML 読込と検証から <code>get_path</code> 、 <code>get_section</code> 、 <code>load_profile as load_workflow_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主な使用位置は L649, L650, L651, L659, L660, L661, L662, L663, ...。
45	<code>from mpc_solarcar.upper_ cost import active_upper_ cost_terms, load_upper_cost_ config, quad_penalty, upper_ stage_cost, upper_terminal_ cost</code>	上位 MPC 目的関数から <code>active_upper_cost_terms</code> 、 <code>load_upper_cost_config</code> 、 <code>quad_penalty</code> 、 <code>upper_stage_cost</code> 、 <code>upper_terminal_cost</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/upper_cost.py</code> 。このファイル内での主な使用位置は L1350, L1360, L1362, L1366, L1370, L1376, L1378, L1379, ...。
52	<code>from mpc_solarcar.upper_ horizon import build_upper_ distance_horizon, plan_ segment_index</code>	上位 MPC 距離メッシュ生成から <code>build_upper_distance_horizon</code> 、 <code>plan_segment_index</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/upper_horizon.py</code> 。このファイル内での主な使用位置は L2341, L3431, L3470, L3590。
53	<code>from mpc_solarcar.upper_ policy import absolute_ control_distances, shift_ upper_policy_warm_start</code>	上位速度計画の補間と <code>warm start</code> から <code>absolute_control_distances</code> 、 <code>shift_upper_policy_warm_start</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/upper_policy.py</code> 。このファイル内での主な使用位置は L2366, L2374。
57	<code>from mpc_solarcar.upper_ solver import hybrid_bounded_ minimize</code>	上位探索ソルバから <code>hybrid_bounded_minimize</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/upper_solver.py</code> 。このファイル内での主な使用位置は L2873。
1394	<code>from scipy. optimize import minimize</code>	連続最適化や MHE の逆推定を解くため。このファイル内での主な使用位置は L1396。

関数・クラスを上から順に解説

L63 関数 `load_yaml`

- 定義: `load_yaml(path)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `open`, `safe_load`
 - 戻り値の要点: `{}` / `yaml.safe_load(f)` or `{}` / `{}`
1. 条件 `not path` を判定し、真なら内部処理を行う。
 2. 例外処理を伴う `try` ブロックを実行する。

L73 関数 `sim_log`

- 定義: `sim_log(message)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `print`
 - 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
1. `print(...)` を実行する。

L77 関数 `select_optimized_vector`

- 定義: `select_optimized_vector(res, x0, label)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `all`, `asarray`, `bool`, `getattr`, `isfinite`, `sim_log`, `str`, `strip`
 - 戻り値の要点: `x_arr` / `x0_arr` / `x0_arr` / `x0_arr`
1. `x0_arr` に `np.asarray(x0, dtype=float)` の結果を代入する。
 2. 条件 `res is None` を判定し、真なら内部処理を行う。
 3. `success` に `bool(getattr(res, 'success', False))` の結果を代入する。
 4. `status` に `getattr(res, 'status', None)` の結果を代入する。
 5. `message` に `str(getattr(res, 'message', '')) or ''` の結果を代入する。
 6. `raw_x` に `getattr(res, 'x', None)` の結果を代入する。
 7. 条件 `raw_x is None` を判定し、真なら内部処理を行う。
 8. 例外処理を伴う `try` ブロックを実行する。
 9. 条件 `x_arr.shape != x0_arr.shape or not np.all(np.isfinite(x_arr))` を判定し、真なら内部処理を行う。
 10. 条件 `not success` を判定し、真なら内部処理を行う。

L115 関数 `limit_step_duration_to_distance`

- 定義: `limit_step_duration_to_distance(step_sec, speed_kmh, s0_kmh, race_kmh)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float, max, min`
- 戻り値の要点: `min(step_sec, max_step_sec) / step_sec / 0.0 / step_sec`

1. `step_sec` に `max(0.0, float(step_sec))` の結果を代入する。
2. 条件 `race_kmh is None` を判定し、真なら内部処理を行う。
3. `remaining_kmh` に `max(0.0, float(race_kmh) - float(s0_kmh))` の結果を代入する。
4. 条件 `remaining_kmh <= DISTANCE_EPS_KMH` を判定し、真なら内部処理を行う。
5. `speed_kmh` に `max(0.0, float(speed_kmh))` の結果を代入する。
6. 条件 `speed_kmh <= 1e-09` を判定し、真なら内部処理を行う。
7. `max_step_sec` に `remaining_kmh * 3600.0 / speed_kmh` の結果を代入する。
8. `min(step_sec, max_step_sec)` を返す。

L129 関数 `terminal_soc_predictions`

- 定義: `terminal_soc_predictions(upper_solve_log)`
- docstring: Return initial full-horizon and latest nontrivial terminal-SoC predictions.
- このブロックが直接呼ぶ主な関数・メソッド: `append, dict, float, get, int, isfinite`
- 戻り値の要点: `(float(initial_soc), float(latest_nontrivial_soc)) / (math.nan, math.nan)`

1. `candidates` に `[]` の結果を代入する。
2. `upper_solve_log` を順に走査し、各要素を `row` に入れて処理する。
3. 条件 `not candidates` を判定し、真なら内部処理を行う。
4. `initial_soc` に `candidates[0][1]` の結果を代入する。
5. `nontrivial` に `[terminal_soc for steps, terminal_soc in candidates if steps > 1]` の結果を代入する。
6. `latest_nontrivial_soc` に `nontrivial[-1]` if `nontrivial` else `candidates[-1][1]` の結果を代入する。
7. `(float(initial_soc), float(latest_nontrivial_soc))` を返す。

L145 関数 `advance_rate_limiter_to_distance_boundary`

- 定義: `advance_rate_limiter_to_distance_boundary(limiter, target_kmh, start_time_sec, step_sec, s0_kmh, distance_limit_kmh)`
- docstring: Advance a limiter by the actual substep, including a short boundary remainder.
- このブロックが直接呼ぶ主な関数・メソッド: `abs, float, limit_step_duration_to_distance, max, range, update`
- 戻り値の要点: `(speed_kmh, actual_dt) / (float(limiter.value), 0.0) / (speed_kmh, next_dt)`

1. `requested_dt` に `max(0.0, float(step_sec))` の結果を代入する。
2. 条件 `requested_dt <= 0.0` を判定し、真なら内部処理を行う。
3. `initial_value` に `float(limiter.value)` の結果を代入する。
4. `actual_dt` に `requested_dt` の結果を代入する。
5. `speed_kmh` に `initial_value` の結果を代入する。
6. `range(8)` を順に走査し、各要素を `_` に入れて処理する。
7. `limiter.value` に `initial_value` の結果を代入する。
8. `limiter.last_time` に `float(start_time_sec)` の結果を代入する。
9. `speed_kmh` に `float(limiter.update(target_kmh, now=float(start_time_sec) + actual_dt))` の結果を代入する。
10. `(speed_kmh, actual_dt)` を返す。

L195 関数 `snap_execution_stop_speed_kmh`

- 定義: `snap_execution_stop_speed_kmh(speed_kmh, target_kmh, stop_requested, deadband_kmh, quantize_step_kmh)`
 - docstring: Remove the final quantized crawl once a stop command has nearly settled.
 - このブロックが直接呼ぶ主な関数・メソッド: `bool, float, max`
 - 戻り値の要点: `(speed, False) / (0.0, True)`
1. `speed` に `max(0.0, float(speed_kmh))` の結果を代入する。
 2. `target` に `max(0.0, float(target_kmh))` の結果を代入する。
 3. `threshold` に `max(1e-06, float(deadband_kmh), float(quantize_step_kmh))` の結果を代入する。
 4. 条件 `bool(stop_requested) and target <= 1e-09 and (speed <= threshold + 1e-09)` を判定し、真なら内部処理を行う。
 5. `(speed, False)` を返す。

L212 関数 `choose_integration_step_seconds`

- 定義: `choose_integration_step_seconds(simulation_step_sec, weather_step_sec)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `float, max, min`
 - 戻り値の要点: `max(60.0, min(simulation_step, weather_step))`
1. `simulation_step` に `float(max(simulation_step_sec, 1.0))` の結果を代入する。
 2. `weather_step` に `float(max(weather_step_sec, 1.0))` の結果を代入する。
 3. `max(60.0, min(simulation_step, weather_step))` を返す。

L218 関数 `choose_stationary_integration_step_seconds`

- 定義: `choose_stationary_integration_step_seconds(simulation_step_sec, weather_step_sec, requested_step_sec)`
 - docstring: Use a fine state step after weather interpolation during long stops.
 - このブロックが直接呼ぶ主な関数・メソッド: `float`, `max`, `min`
 - 戻り値の要点: `min(requested, simulation_step, weather_step)`
1. `requested` に `max(1.0, float(requested_step_sec))` の結果を代入する。
 2. `simulation_step` に `max(1.0, float(simulation_step_sec))` の結果を代入する。
 3. `weather_step` に `max(1.0, float(weather_step_sec))` の結果を代入する。
 4. `min(requested, simulation_step, weather_step)` を返す。

L230 関数 `write_model_snapshot`

- 定義: `write_model_snapshot(detail_csv, parameters, map_paths)`
 - docstring: Write immutable model provenance once instead of repeating map paths at 1 Hz.
 - このブロックが直接呼ぶ主な関数・メソッド: `Path`, `bool`, `dict`, `dumps`, `encode`, `endswith`, `ensure_parent_dir`, `hexdigest`, `int`, `is_file`, `items`, `iter`
 - 戻り値の要点: `(snapshot_id, output)`
1. `maps` に `{}` の結果を代入する。
 2. `sorted((map_paths or {}).items())` を順に走査し、各要素を `(key, raw_path)` に入れて処理する。
 3. `canonical` に `{'parameters': dict(sorted((parameters or {}).items()))}`, `'maps': maps}` の結果を代入する。
 4. `snapshot_id` に `hashlib.sha256(json.dumps(canonical, sort_keys=True, ensure_ascii=False, default=str).encode('utf-8')).hexdigest()[:16]` の結果を代入する。
 5. `raw_base` に `str(detail_csv)` の結果を代入する。
 6. 条件 `raw_base.lower().endswith('.gz')` を判定し、真なら内部処理を行う。
 7. 条件 `raw_base.lower().endswith('.csv')` を判定し、真なら内部処理を行う。
 8. `output` に `raw_base + '_model_snapshot.yaml'` の結果を代入する。
 9. `payload` に `{'schema_version': 1, 'model_snapshot_id': snapshot_id, 'detail_csv': str(detail_csv), 'parameters_repeated_in_detail_csv': True, 'maps_referenced_by_snapshot_id': True, **canonical}` の結果を代入する。
 10. `ensure_parent_dir(...)` を実行する。

L278 関数 `get_workspace_revision`

- 定義: `get_workspace_revision(root_dir)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `bool`, `lower`, `run`, `strip`
 - 戻り値の要点: `info / info / info / info`
1. `info` に `{'git_available': False, 'git_head': '', 'git_branch': '', 'git_dirty': None}` の結果を代入する。
 2. 例外処理を伴う `try` ブロックを実行する。
 3. 条件 `inside.returncode != 0 or inside.stdout.strip().lower() != 'true'` を判定し、真なら内部処理を行う。
 4. `info['git_available']` に `True` の結果を代入する。
 5. 例外処理を伴う `try` ブロックを実行する。
 6. `info` を返す。

L331 関数 `timestamp_ns`

- 定義: `timestamp_ns(value)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `int`
 - 戻り値の要点: `int(pd.Timestamp(value).value)`
1. `int(pd.Timestamp(value).value)` を返す。

L335 関数 `load_stops`

- 定義: `load_stops(path)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `append`, `float`, `get`, `isinstance`, `load_yaml`, `max`, `str`
 - 戻り値の要点: `stops`
1. `cfg` に `load_yaml(path)` の結果を代入する。
 2. `raw_stops` に `cfg.get('stops', []) if isinstance(cfg, dict) else []` の結果を代入する。
 3. `stops` に `[]` の結果を代入する。
 4. `raw_stops` or `[]` を順に走査し、各要素を `item` に入れて処理する。
 5. `stops` を返す。

L365 関数 load_profile

- 定義: load_profile(path)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: read_csv
- 戻り値の要点: None / pd.read_csv(path) / None

1. 条件 not path を判定し、真なら内部処理を行う。
2. 例外処理を伴う try ブロックを実行する。

L373 関数 forecast_distance_column

- 定義: forecast_distance_column(df)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: isinstance
- 戻り値の要点: " / " / 's_km' / 'route_progress_km'

1. 条件 not isinstance(df, pd.DataFrame) を判定し、真なら内部処理を行う。
2. 条件's_km' in df.columns を判定し、真なら内部処理を行う。
3. 条件'route_progress_km' in df.columns を判定し、真なら内部処理を行う。
4. " を返す。

L383 関数 load_forecast_dataframe

- 定義: load_forecast_dataframe(path, tzname)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: ZoneInfo, copy, drop_duplicates, forecast_distance_column, notna, read_csv, reset_index, sort_values, str, to_datetime, tz_convert, tz_localize
- 戻り値の要点: df / df

1. df に pd.read_csv(path) の結果を代入する。
2. 条件'time' not in df.columns を判定し、真なら内部処理を行う。
3. t に pd.to_datetime(df['time'], format='mixed', errors='coerce') の結果を代入する。
4. tzname に str(tzname or 'UTC') の結果を代入する。
5. 条件 t.dt.tz is None を判定し、真なら内部処理を行う。
6. df に df.copy() の結果を代入する。
7. df['time'] に t の結果を代入する。
8. dist_col に forecast_distance_column(df) の結果を代入する。
9. sort_cols に ['time'] + ([dist_col] if dist_col else []) の結果を代入する。
10. dedup_cols に ['time'] + ([dist_col] if dist_col else []) の結果を代入する。

L410 関数 `merge_forecast_dataframes`

- 定義: `merge_forecast_dataframes(primary, fallback)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `combine_first`, `copy`, `len`, `reset_index`, `set_index`, `sort_index`
- 戻り値の要点: `merged / fallback.copy() if fallback is not None else pd.DataFrame() / primary.copy() / primary.copy()`

1. 条件 `primary is None or len(primary) == 0` を判定し、真なら内部処理を行う。
2. 条件 `fallback is None or len(fallback) == 0` を判定し、真なら内部処理を行う。
3. 条件 `'time' not in primary.columns or 'time' not in fallback.columns` を判定し、真なら内部処理を行う。
4. `primary_idx` に `primary.set_index('time')` の結果を代入する。
5. `fallback_idx` に `fallback.set_index('time')` の結果を代入する。
6. `merged` に `primary_idx.combine_first(fallback_idx).sort_index().reset_index()` の結果を代入する。
7. `merged` を返す。

L423 関数 `build_forecast_grid_payload`

- 定義: `build_forecast_grid_payload(df)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Index`, `apply`, `array`, `bfill`, `copy`, `drop_duplicates`, `dropna`, `ffill`, `forecast_distance_column`, `interpolate`, `len`, `max`
- 戻り値の要点: `{'dist_col': dist_col, 'time_ns': np.array([timestamp_ns(value) for value in time_index], dtype=np.int64), 's_grid': s_grid, 'matrices': matrices} / None / None / None`

1. `dist_col` に `forecast_distance_column(df)` の結果を代入する。
2. 条件 `not dist_col or 'time' not in df.columns` を判定し、真なら内部処理を行う。
3. `work` に `df.copy()` の結果を代入する。
4. `work[dist_col]` に `pd.to_numeric(work[dist_col], errors='coerce')` の結果を代入する。
5. `work` に `work.dropna(subset=['time', dist_col]).sort_values(['time', dist_col])` の結果を代入する。
6. 条件 `work.empty` を判定し、真なら内部処理を行う。
7. `time_index` に `pd.Index(work['time'].drop_duplicates().sort_values())` の結果を代入する。
8. `s_grid` に `np.array(sorted(work[dist_col].dropna().unique()), dtype=float)` の結果を代入する。
9. 条件 `len(time_index) < 2 or len(s_grid) < 2` を判定し、真なら内部処理を行う。
10. 条件 `len(work) <= max(len(time_index), len(s_grid))` を判定し、真なら内部処理を行う。

L476 関数 `interp_forecast_grid`

- 定義: `interp_forecast_grid(payload, col, t_utc, s_km, default)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `clip`, `float`, `get`, `int`, `len`, `max`, `searchsorted`, `timestamp_ns`
- 戻り値の要点: `float((1.0 - wt) * v0 + wt * v1) / float(default) / float(default)`

1. 条件 `not payload` を判定し、真なら内部処理を行う。
2. `matrix` に `payload.get('matrices', {})`.`get(col)` の結果を代入する。
3. `tg` に `payload.get('time_ns')` の結果を代入する。
4. `sg` に `payload.get('s_grid')` の結果を代入する。
5. 条件 `matrix is None or tg is None or sg is None or (len(tg) == 0) or (len(sg) == 0)` を判定し、真なら内部処理を行う。
6. `t_ns` に `int(np.clip(timestamp_ns(t_utc), int(tg[0]), int(tg[-1])))` の結果を代入する。
7. `s_val` に `float(s_km if s_km is not None else sg[0])` の結果を代入する。
8. `s_val` に `float(np.clip(s_val, float(sg[0]), float(sg[-1])))` の結果を代入する。
9. `i_hi` に `int(np.searchsorted(tg, t_ns, side='left'))` の結果を代入する。
10. 条件 `i_hi <= 0` を判定し、真なら内部処理を行う。

L523 関数 `load_progress_reference_dataframe`

- 定義: `load_progress_reference_dataframe(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `drop_duplicates`, `dropna`, `read_csv`, `reset_index`, `sort_values`, `to_datetime`, `to_numeric`
- 戻り値の要点: `out.reset_index(drop=True) / None / None / None`

1. 条件 `not path` を判定し、真なら内部処理を行う。
2. 例外処理を伴う `try` ブロックを実行する。
3. `time_col` に `'time_utc' if 'time_utc' in df.columns else 'time' if 'time' in df.columns else ''` の結果を代入する。
4. 条件 `not time_col or 's_km' not in df.columns` を判定し、真なら内部処理を行う。
5. `t` に `pd.to_datetime(df[time_col], format='mixed', utc=True, errors='coerce')` の結果を代入する。
6. `out` に `pd.DataFrame({'time_utc': t, 's_km': pd.to_numeric(df['s_km'], errors='coerce')})` の結果を代入する。
7. 条件 `'speed_kmh' in df.columns` を判定し、真なら内部処理を行う。
8. `out` に `out.dropna(subset=['time_utc', 's_km']).sort_values('time_utc').drop_duplicates(subset=['time_utc'], keep='last')` の結果を代入する。
9. 条件 `out.empty` を判定し、真なら内部処理を行う。
10. `out.reset_index(drop=True)` を返す。

L543 関数 `ensure_parent_dir`

- 定義: `ensure_parent_dir(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dirname`, `makedirs`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. `parent` に `os.path.dirname(path)` の結果を代入する。
2. 条件 `parent` を判定し、真なら内部処理を行う。

L549 クラス `DetailCsvStream`

- 定義: `DetailCsvStream(bases=)`
- docstring: Write the 1 Hz execution trace without retaining the full race in RAM.
- このブロックが直接呼ぶ主な関数・メソッド: `DictWriter`, `close`, `endswith`, `ensure_parent_dir`, `flush`, `keys`, `list`, `lower`, `opener`, `str`, `writeheader`, `writerow`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. docstring または説明文字列を置く。
2. 関数 `__init__` を定義する。
3. 関数 `write` を定義する。
4. 関数 `close` を定義する。
5. 関数 `__enter__` を定義する。
6. 関数 `__exit__` を定義する。

L583 関数 `_deep_copy_cfg`

- 定義: `_deep_copy_cfg(cfg)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `deepcopy`, `isinstance`
- 戻り値の要点: `copy.deepcopy(cfg) if isinstance(cfg, dict) else {}`

1. `copy.deepcopy(cfg) if isinstance(cfg, dict) else {}` を返す。

L587 関数 `_set_nested`

- 定義: `_set_nested(cfg, dotted_key, value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `get`, `isinstance`, `split`, `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. `parts` に `[part for part in str(dotted_key).split('.') if part]` の結果を代入する。

2. 条件 `not parts` を判定し、真なら内部処理を行う。
3. `cur` に `cfg` の結果を代入する。
4. `parts[:-1]` を順に走査し、各要素を `part` に入れて処理する。
5. `cur[parts[-1]]` に `value` の結果を代入する。

L601 関数 `apply_overrides`

- 定義: `apply_overrides(cfg, overrides)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `_deep_copy_cfg`, `_set_nested`, `append`, `safe_load`, `split`, `str`, `strip`
 - 戻り値の要点: `(cfg, applied)`
1. `cfg` に `_deep_copy_cfg(cfg)` の結果を代入する。
 2. `applied` に `[]` の結果を代入する。
 3. `overrides` or `[]` を順に走査し、各要素を `raw` に入れて処理する。
 4. `(cfg, applied)` を返す。

L618 関数 `build_config_tag`

- 定義: `build_config_tag(profile_path, profile_cfg, overrides)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `encode`, `fromtimestamp`, `getmtime`, `hexdigest`, `isinstance`, `now`, `safe_dump`, `sha1`, `strftime`
 - 戻り値の要点: `f'{stamp}_{digest}'`
1. `payload` に `{'profile_cfg': profile_cfg if isinstance(profile_cfg, dict) else {}, 'overrides': overrides or []}` の結果を代入する。
 2. `canonical` に `yaml.safe_dump(payload, sort_keys=True, allow_unicode=True)` の結果を代入する。
 3. `digest` に `hashlib.sha1(canonical.encode('utf-8')).hexdigest()[8:]` の結果を代入する。
 4. 例外処理を伴う `try` ブロックを実行する。
 5. `stamp` に `mtime.strftime('%Y%m%d_%H%M%S')` の結果を代入する。
 6. `f'{stamp}_{digest}'` を返す。

L633 関数 `tag_output_path`

- 定義: `tag_output_path(path_value, tag, default_ext)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `replace`, `splitext`, `str`, `strip`
- 戻り値の要点: `f'{stem}_{tag}{ext}' / raw / raw.replace('{tag}', tag)`

1. raw に `str(path_value or "").strip()` の結果を代入する。
2. 条件 `not raw or not tag` を判定し、真なら内部処理を行う。
3. 条件 `'{tag}' in raw` を判定し、真なら内部処理を行う。
4. (stem, ext) に `os.path.splitext(raw)` の結果を代入する。
5. 条件 `not ext and default_ext` を判定し、真なら内部処理を行う。
6. `f'{stem}_{tag}{ext}'` を返す。

L645 関数 `apply_profile_cfg_to_args`

- 定義: `apply_profile_cfg_to_args(profile_path, profile_cfg, args, force_output_defaults)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `bool, build_config_tag, clip, endswith, float, get, get_path, get_section, getattr, int, isinstance, join`
 - 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
1. 条件 `not profile_cfg` を判定し、真なら内部処理を行う。
 2. `sim_cfg` に `get_section(profile_cfg, 'simulation')` の結果を代入する。
 3. `runtime_cfg` に `get_section(profile_cfg, 'runtime')` の結果を代入する。
 4. `logging_cfg` に `get_section(profile_cfg, 'logging')` の結果を代入する。
 5. `auto_version_outputs` に `bool(sim_cfg.get('auto_version_outputs', True))` の結果を代入する。
 6. `output_tag` に `build_config_tag(profile_path, profile_cfg, getattr(args, 'override', []))` if `auto_version_outputs` else ” の結果を代入する。
 7. `args.params_yaml` に `profile_path` の結果を代入する。
 8. `args.profile_path_resolved` に `profile_path` の結果を代入する。
 9. `args.auto_version_outputs` に `auto_version_outputs` の結果を代入する。
 10. `args.output_tag` に `output_tag` の結果を代入する。

L798 関数 `apply_profile_defaults`

- 定義: `apply_profile_defaults(args)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `apply_profile_cfg_to_args, load_workflow_profile`
 - 戻り値の要点: `profile_cfg / {}`
1. 条件 `not args.profile_yaml` を判定し、真なら内部処理を行う。
 2. (`profile_path, profile_cfg`) に `load_workflow_profile(args.profile_yaml)` の結果を代入する。
 3. `apply_profile_cfg_to_args(...)` を実行する。
 4. `profile_cfg` を返す。

L806 関数 `resolve_config_path`

- 定義: `resolve_config_path(profile_path, value)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Path`, `exists`, `is_absolute`, `resolve`, `str`, `strip`
 - 戻り値の要点: `str((ROOT / path).resolve()) / "" / str(path) / str(candidate)`
1. `value` に `str(value or "").strip()` の結果を代入する。
 2. 条件 `not value` を判定し、真なら内部処理を行う。
 3. `path` に `Path(value)` の結果を代入する。
 4. 条件 `path.is_absolute()` を判定し、真なら内部処理を行う。
 5. `profile_dir` に `Path(profile_path).resolve().parent if profile_path else ROOT` の結果を代入する。
 6. `candidate` に `(profile_dir / path).resolve()` の結果を代入する。
 7. 条件 `candidate.exists()` を判定し、真なら内部処理を行う。
 8. `str((ROOT / path).resolve())` を返す。

L820 関数 `evaluate_model_validation_gate`

- 定義: `evaluate_model_validation_gate(cfg, profile_path)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Path`, `abs`, `all`, `bool`, `dict`, `exists`, `float`, `get`, `int`, `isfinite`, `isinstance`, `load_yaml`
 - 戻り値の要点: `result / result`
1. `identification` に `cfg.get('identification', {})` if `isinstance(cfg, dict)` else `{}` の結果を代入する。
 2. `gate_cfg` に `identification.get('validation_gate', {})` if `isinstance(identification, dict)` else `{}` の結果を代入する。
 3. 条件 `not isinstance(gate_cfg, dict)` を判定し、真なら内部処理を行う。
 4. `fit_summary_path` に `resolve_config_path(profile_path, identification.get('fit_summary_yaml', ""))` の結果を代入する。
 5. `terminal_path` に `resolve_config_path(profile_path, identification.get('terminal_consistency_yaml', ""))` の結果を代入する。
 6. `result` に `{'available': False, 'fit_summary_yaml': fit_summary_path, 'terminal_consistency_yaml': terminal_path, 'gate_pass': False, 'checks': {}, 'thresholds': {}}` の結果を代入する。
 7. 条件 `not fit_summary_path or not Path(fit_summary_path).exists()` を判定し、真なら内部処理を行う。
 8. `summary` に `load_yaml(fit_summary_path)` の結果を代入する。
 9. `metrics` に `summary.get('validation_metrics', {})` if `isinstance(summary, dict)` else `{}` の結果を代入する。

10. thresholds に {'vehicle_power_rmse_max_w': float(gate_cfg.get('vehicle_power_rmse_max_w', 150.0)), 'vehicle_voltage_rmse_max_v': float(gate_cfg.get('vehicle_voltage_rmse_max_v', 1.0)), 'conditional_power_rmse_max_w': float(gate_cfg.get('conditional_power_rmse_max_w', 150.0)), 'conditional_voltage_rmse_max_v': float(gate_cfg.get('conditional_voltage_rmse_max_v', 1.0)), 'end_to_end_power_rmse_max_w': float(gate_cfg.get('end_to_end_power_rmse_max_w', 200.0)), 'end_to_end_voltage_rmse_max_v': float(gate_cfg.get('end_to_end_voltage_rmse_max_v', 2.0)), 'moving_pv_rmse_max_w': float(gate_cfg.get('moving_pv_rmse_max_w', 150.0)), 'pv_lodo_moving_rmse_max_w': float(gate_cfg.get('pv_lodo_moving_rmse_max_w', 150.0)), 'pv_lodo_deployed_rmse_max_w': float(gate_cfg.get('pv_lodo_deployed_rmse_max_w', 200.0)), 'power_residual_mean_120s_rmse_max_w': float(gate_cfg.get('power_residual_mean_120s_rmse_max_w', 150.0)), 'energy_error_25km_rmse_max_wh': float(gate_cfg.get('energy_error_25km_rmse_max_wh', 35.0)), 'terminal_soc_evidence_spread_max': float(gate_cfg.get('terminal_soc_evidence_spread_max', 0.05)), 'terminal_replay_soc_error_max': float(gate_cfg.get('terminal_replay_soc_error_max', 0.02)), 'terminal_replay_voltage_error_max_v': float(gate_cfg.get('terminal_replay_voltage_error_max_v', 0.5)), 'vehicle_terminal_soc_error_max': float(gate_cfg.get('vehicle_terminal_soc_error_max', 0.02)), 'vehicle_terminal_voltage_error_max_v': float(gate_cfg.get('vehicle_terminal_voltage_error_max_v', 0.5)), 'end_to_end_terminal_soc_error_max': float(gate_cfg.get('end_to_end_terminal_soc_error_max', 0.03)), 'end_to_end_terminal_voltage_error_max_v': float(gate_cfg.get('end_to_end_terminal_voltage_error_max_v', 1.0)), 'acceleration_validation_rmse_ratio_max': float(gate_cfg.get('acceleration_validation_rmse_ratio_max', 1.02)), 'acceleration_validation_min_samples': int(gate_cfg.get('acceleration_validation_min_samples', 100)), 'grade_validation_rmse_ratio_max': float(gate_cfg.get('grade_validation_rmse_ratio_max', 1.02)), 'grade_validation_min_samples': int(gate_cfg.get('grade_validation_min_samples', 100))} の結果を代入する。

L1032 関数 get_profile_val

- 定義: `get_profile_val(df, s_km, field, default)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `float`, `interpolate_profile`, `isfinite`
 - 戻り値の要点: `float(val) / float(default) / float(default)`
1. 条件 `df is None or field not in df.columns` を判定し、真なら内部処理を行う。
 2. `val` に `float(interpolate_profile(df, s_km, field, default))` の結果を代入する。
 3. 条件 `not math.isfinite(val)` を判定し、真なら内部処理を行う。
 4. `float(val)` を返す。

L1040 関数 parse_utc_arg

- 定義: `parse_utc_arg(raw_value)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `astimezone`, `endswith`, `fromisoformat`, `replace`, `str`, `strip`
 - 戻り値の要点: `dt.astimezone(timezone.utc) / None`
1. `text` に `str(raw_value or '').strip()` の結果を代入する。
 2. 条件 `not text` を判定し、真なら内部処理を行う。
 3. 条件 `text.endswith('Z')` を判定し、真なら内部処理を行う。
 4. `dt` に `datetime.fromisoformat(text)` の結果を代入する。
 5. 条件 `dt.tzinfo is None` を判定し、真なら内部処理を行う。
 6. `dt.astimezone(timezone.utc)` を返す。

L1052 関数 `resolve_forecast_mode`

- 定義: `resolve_forecast_mode(mode, df)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `all`, `isna`, `lower`, `str`, `strip`
 - 戻り値の要点: `raw / 'absolute' if has_time else 'relative' / 'relative' / 'relative'`
1. `raw` に `str(mode or 'auto').strip().lower()` の結果を代入する。
 2. `has_time` に `'time' in df.columns and (not df['time'].isna().all())` の結果を代入する。
 3. 条件 `raw == 'auto'` を判定し、真なら内部処理を行う。
 4. 条件 `raw not in ('absolute', 'relative', 'loop')` を判定し、真なら内部処理を行う。
 5. 条件 `raw == 'absolute' and (not has_time)` を判定し、真なら内部処理を行う。
 6. `raw` を返す。

L1064 関数 `forecast_row_index`

- 定義: `forecast_row_index(df, sim_t, dt_sec, mode, forecast_start_time)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `all`, `clip`, `int`, `isna`, `len`, `max`, `searchsorted`, `total_seconds`
 - 戻り値の要点: `int(np.clip(idx, 0, len(df) - 1)) / 0 / int(idx % len(df)) / int(np.clip(idx, 0, len(df) - 1))`
1. 条件 `len(df) == 0` を判定し、真なら内部処理を行う。
 2. 条件 `mode == 'absolute' and 'time' in df.columns and (not df['time'].isna().all())` を判定し、真なら内部処理を行う。
 3. `elapsed` に `max(0.0, (sim_t - forecast_start_time).total_seconds())` の結果を代入する。
 4. `idx` に `int(elapsed / max(dt_sec, 0.001))` の結果を代入する。
 5. 条件 `mode == 'loop'` を判定し、真なら内部処理を行う。
 6. `int(np.clip(idx, 0, len(df) - 1))` を返す。

L1082 関数 `format_metric`

- 定義: `format_metric(value, digits, default)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`
 - 戻り値の要点: `default / default / f'{fval:.{digits}f}'`
1. 条件 `value is None` を判定し、真なら内部処理を行う。
 2. 例外処理を伴う `try` ブロックを実行する。
 3. `default` を返す。

L1094 関数 `decimate_xy`

- 定義: `decimate_xy(xs,ys,max_points)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `append`, `ceil`, `float`, `int`, `isfinite`, `len`, `max`, `zip`
 - 戻り値の要点: `reduced / pts`
1. `pts` に `[(float(x), float(y)) for x, y in zip(xs, ys) if math.isfinite(float(y))]` の結果を代入する。
 2. 条件 `len(pts) <= max_points` を判定し、真なら内部処理を行う。
 3. `step` に `max(1, int(math.ceil(len(pts) / max_points)))` の結果を代入する。
 4. `reduced` に `pts[::step]` の結果を代入する。
 5. 条件 `reduced[-1] != pts[-1]` を判定し、真なら内部処理を行う。
 6. `reduced` を返す。

L1105 関数 `build_svg_chart`

- 定義: `build_svg_chart(xs,ys,color,width,height,pad,label)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `append`, `decimate_xy`, `escape`, `format_metric`, `join`, `max`, `min`
- 戻り値の要点: `f'<svg viewBox="0 0 {width} {height}" class="chart-svg" role="img" aria-label="{html.escape(label)}">x="0" y="0" width="{width}" height="{height}" rx="18" ry="18" fill="#ffdf8" stroke="#d8d1c4" /><line x1="{pad}" y1="{pad}" x2="{pad}" y2="{height - pad}" stroke="#a9a293" stroke-width="1" /><line x1="{pad}" y1="{height - pad}" x2="{width - pad}" y2="{height - pad}" stroke="#a9a293" stroke-width="1" /><polyline fill="none" stroke="{color}" stroke-width="2.6" points="{polyline}" /><text x="{pad}" y="18" font-size="12" fill="#50483f">{html.escape(label)}</text><text x="{width - pad}" y="18" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(y_max, 2)}</text><text x="{width - pad}" y="{height - 8}" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(x_max, 1)}</text><text x="{pad}" y="{height - 8}" font-size="11" fill="#6f665b">{format_metric(x_min, 1)}</text><text x="{width - pad}" y="{height / 2:.1f}" text-anchor="end" font-size="11" fill="#6f665b">{format_metric(y_mid, 2)}</text><text x="{width - pad}" y="{height - pad + 16}" text-anchor="end" font-size="11" fill="#6f665b">x</text></svg>' /><div class="chart-empty">no data</div>`

1. `pts` に `decimate_xy(xs, ys, max_points=700)` の結果を代入する。
2. 条件 `not pts` を判定し、真なら内部処理を行う。
3. `x_vals` に `[p[0] for p in pts]` の結果を代入する。
4. `y_vals` に `[p[1] for p in pts]` の結果を代入する。
5. `x_min` に `min(x_vals)` の結果を代入する。
6. `x_max` に `max(x_vals)` の結果を代入する。
7. `y_min` に `min(y_vals)` の結果を代入する。
8. `y_max` に `max(y_vals)` の結果を代入する。
9. 条件 `x_max <= x_min` を判定し、真なら内部処理を行う。
10. 条件 `y_max <= y_min` を判定し、真なら内部処理を行う。

L1146 関数 `flatten_params_for_report`

- 定義: `flatten_params_for_report(cfg)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `append`, `get`, `isinstance`, `items`, `str`, `visit`
 - 戻り値の要点: `rows / rows`
1. `rows` に `[]` の結果を代入する。
 2. 関数 `visit` を定義する。
 3. 条件 `not isinstance(cfg, dict)` を判定し、真なら内部処理を行う。
 4. `('simulation', 'model', 'mpc', 'runtime')` を順に走査し、各要素を `section_name` に入れて処理する。
 5. `rows` を返す。

L1166 関数 `write_simulation_report`

- 定義: `write_simulation_report(path, summary, detail_df, params_rows)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Series`, `append`, `build_svg_chart`, `dumps`, `ensure_parent_dir`, `escape`, `extend`, `format_metric`, `get`, `join`, `len`, `list`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `ensure_parent_dir(...)` を実行する。
 2. `x_index` に `list(range(len(detail_df)))` の結果を代入する。
 3. `speed_exec_present` に `'v_exec_kmh' in detail_df.columns` の結果を代入する。
 4. `speed_series` に `detail_df.get('v_exec_kmh', detail_df.get('v_cmd_kmh', pd.Series(dtype=float)))` の結果を代入する。
 5. `speed_svg` に `build_svg_chart(x_index, speed_series, color='#0f766e', label='speed exec [km/h]' if speed_exec_present else 'speed cmd [km/h]')` の結果を代入する。
 6. `speed_cmd_svg` に `''` の結果を代入する。
 7. 条件 `speed_exec_present` を判定し、真なら内部処理を行う。
 8. `soc_svg` に `build_svg_chart(x_index, detail_df.get('soc', pd.Series(dtype=float)), color='#b45309', label='soc [-]')` の結果を代入する。
 9. `pack_svg` に `build_svg_chart(x_index, detail_df.get('P_pack', pd.Series(dtype=float)), color='#b91c1c', label='pack power [W]')` の結果を代入する。
 10. `solar_svg` に `build_svg_chart(x_index, detail_df.get('P_pv', pd.Series(dtype=float)), color='#2563eb', label='pv power [W]')` の結果を代入する。

L1288 関数 `mpc_solve`

- 定義: `mpc_solve(model, data, z0, Tb0, s0_kmh, v0_kmh, v_max_kmh, term_soc_min, w_dv, w_dv_limit, dv_max_kmhps, w_T, w_speed_limit, w_current, speed_profile, soc_target, soc_band, w_soc_target, w_soc_band, schedule, soc_day_end_max, w_soc_day_max, soc_finish_target, soc_finish_tol, w_soc_progress, w_soc_terminal, race_kmh, soc_day_end_target, soc_day_end_tol, w_soc_day_track)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `array`, `clip`, `current_drive_window`, `dict`, `electrical_balance`, `float`, `get`, `get_profile_val`, `is_drive_time`, `len`, `list`
- 戻り値の要点: `float(np.clip(v0_kmh, 0.0, v_max_kmh)) / v0_kmh / x * x / J`

1. `p` に `model.p` の結果を代入する。
2. `Np` に `len(data)` の結果を代入する。
3. 条件 `Np <= 0` を判定し、真なら内部処理を行う。
4. `v0_ms` に `v0_kmh / 3.6` の結果を代入する。
5. `x0` に `np.array([v0_ms] * Np, dtype=float)` の結果を代入する。
6. `lb` に `np.zeros(Np, dtype=float)` の結果を代入する。
7. `ub` に `np.ones(Np, dtype=float) * (v_max_kmh / 3.6)` の結果を代入する。
8. `dv_max_msp` に `dv_max_kmhps / 3.6` の結果を代入する。
9. 関数 `quad_penalty` を定義する。
10. 関数 `cost` を定義する。

L1402 関数 `soc_guard_speed`

- 定義: `soc_guard_speed(model, v_kmh, z, Tb, s_kmh, d0, route_profile, mode, soc_guard)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `electrical_balance`, `float`, `get`, `get_profile_val`, `lower`, `max`, `range`, `soc_step`, `str`, `z_next_for`
- 戻り値の要点: `float(lo) / model.soc_step(z, P_pack, model.p.dt, current_a=float(out['I']), Tbat_C=Tb) / float(lo) / v_kmh`

1. `mode` に `str(mode).lower()` の結果を代入する。
2. `target` に `model.p.soc_min + soc_guard` の結果を代入する。
3. `slope_pct` に `get_profile_val(route_profile, s_kmh, 'slope_pct', d0.get('slope_pct', 0.0))` の結果を代入する。
4. `headwind_ms` に `d0.get('headwind_ms', 0.0)` の結果を代入する。
5. `elevation_m` に `get_profile_val(route_profile, s_kmh, 'elev_m', d0.get('elevation_m', 0.0))` の結果を代入する。
6. 関数 `z_next_for` を定義する。
7. 条件 `z <= target` を判定し、真なら内部処理を行う。
8. 条件 `z_next_for(v_kmh) >= target` を判定し、真なら内部処理を行う。
9. `lo` に `0.0` の結果を代入する。
10. `hi` に `max(0.0, float(v_kmh))` の結果を代入する。

L1451 関数 soc_day_guard_speed

- 定義: `soc_day_guard_speed(model, v_kmh, z, Tb, s_km, d0, route_profile, target_soc, tol)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `electrical_balance`, `float`, `get`, `get_profile_val`, `max`, `range`, `soc_step`, `z_next_for`
- 戻り値の要点: `float(lo) / v_kmh / model.soc_step(z, P_pack, model.p.dt, current_a=float(out['I']), Tbat_C=Tb) / v_kmh`

1. 条件 `target_soc <= 0.0` を判定し、真なら内部処理を行う。
2. `slope_pct` に `get_profile_val(route_profile, s_km, 'slope_pct', d0.get('slope_pct', 0.0))` の結果を代入する。
3. `headwind_ms` に `d0.get('headwind_ms', 0.0)` の結果を代入する。
4. `elevation_m` に `get_profile_val(route_profile, s_km, 'elev_m', d0.get('elevation_m', 0.0))` の結果を代入する。
5. 関数 `z_next_for` を定義する。
6. 条件 `z_next_for(v_kmh) >= target_soc - tol` を判定し、真なら内部処理を行う。
7. `lo` に `0.0` の結果を代入する。
8. `hi` に `max(0.0, float(v_kmh))` の結果を代入する。
9. `range(25)` を順に走査し、各要素を `_` に入れて処理する。
10. `float(lo)` を返す。

L1485 関数 main

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `DataFrame`, `DetailCsvStream`, `Params`, `Path`, `SmoothRateLimiter`, `SolarCarModel`, `Timestamp`, `ValueError`, `ZoneInfo`, `_deep_copy_cfg`, `abs`
- 戻り値の要点: `choose_integration_step_seconds(args.dt, forecast_native_dt_sec) / t_utc >= forecast_start_time / total_sec / s0 + alpha * (s1 - s0)`

1. `ap` に `argparse.ArgumentParser()` の結果を代入する。
2. `ap.add_argument(...)` を実行する。
3. `ap.add_argument(...)` を実行する。
4. `ap.add_argument(...)` を実行する。
5. `ap.add_argument(...)` を実行する。
6. `ap.add_argument(...)` を実行する。
7. `ap.add_argument(...)` を実行する。
8. `ap.add_argument(...)` を実行する。
9. `ap.add_argument(...)` を実行する。
10. `ap.add_argument(...)` を実行する。

CLI 引数

行	引数
1487	--profile_yaml
1488	--forecast_csv
1489	--forecast_fill_csv
1490	--progress_reference_csv
1491	--forecast_time_mode
1492	--forecast_time_tz
1493	--route_profile_csv
1494	--speed_profile_csv
1495	--params_yaml
1496	--stop_yaml
1497	--drive_schedule_yaml
1498	--panel_eff_map
1499	--mppt_eff_map
1500	--ocv_soc_map
1501	--drive_eff_map
1502	--regen_eff_map
1503	--rint_map
1504	--drive_map_eco
1505	--drive_map_power
1506	--regen_map_eco
1507	--regen_map_power
1508	--dt
1509	--horizon_steps
1510	--soc0
1511	--Tb0
1512	--v0_kmh
1513	--start_utc
1514	--forecast_start_time_utc
1515	--start_index
1516	--start_s_km
1517	--resume_csv
1518	--resume_s_km
1519	--out_csv
1520	--out_detail_csv
1521	--report_html
1522	--summary_json
1523	--resolved_yaml
1524	--latest_manifest_json
1525	--override
1526	--soc_guard_margin
1527	--soc_guard_mode
1528	--solar_gain
1529	--poa_gain_drive
1530	--poa_gain_stop

```
1531 --stop_tilt_fraction
1532 --control_stop_tilt_fraction
1533 --energy_budget
1534 --exec_model_enabled
1535 --exec_inner_dt_sec
1536 --detail_rate_hz
1537 --exec_tau_sec
1538 --exec_accel_limit_kmhps
1539 --exec_decel_limit_kmhps
1540 --exec_deadband_kmh
1541 --exec_quantize_step_kmh
1542 --exec_reaction_delay_sec
1543 --upper_mode
1544 --upper_ds_km
1545 --upper_horizon_km
1546 --upper_max_steps
1547 --upper_horizon_mode
1548 --upper_adaptive_min_ds_km
1549 --upper_adaptive_max_ds_km
1550 --upper_adaptive_growth
1551 --upper_replan_km
1552 --upper_replan_sec
1553 --upper_max_iter
1554 --upper_global_search_enabled
1555 --upper_global_search_mode
1556 --upper_cem_generations
1557 --upper_cem_population
1558 --upper_cem_elite
1559 --upper_local_refine_topk
1560 --upper_shgo_samples
1561 --upper_shgo_iters
1562 --upper_cert_grid_levels
1563 --upper_cert_max_evaluations
1564 --upper_cert_workers
1570 --upper_ctrl_km
1571 --upper_vmin_kmh
```

処理の流れ

1. CLI 引数を解釈し、main() から処理を起動する。

このファイルの位置づけ

この資料群では、scripts/solar_sim.py を **offline core** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の profile / launch / telemetry と後段の planner / logger / report へ繋がる。